Cortex-M3

Author: Brian Kang
Create: 5/4/2026
Modified: 5/8/2026

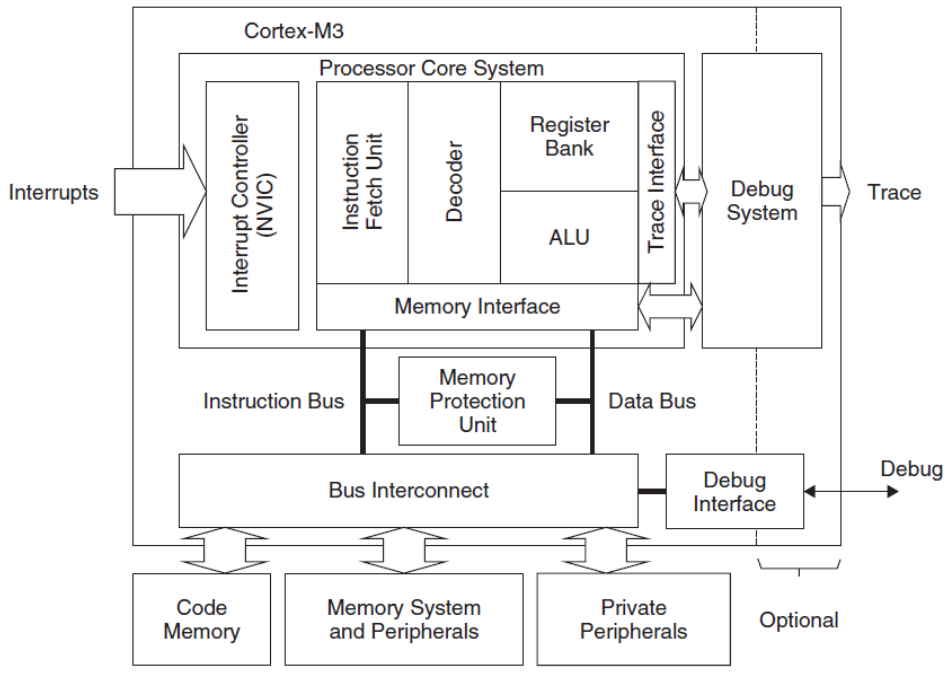
Preface

This application is for Cortex-M3 and to explain basic concepts.
This application note is based “The Definitive Guide to the ARM Cortex-M3”.

ARM Cortex has 3 groups. It will explain Cortex-M.

Cortex-A	Applications, with MMU
Cortex-R	Real-time
Cortex-M	Microcontroller

Cortex-M3 block diagram.



Cortex-M3 Feature

32bit
3-stage pipeline

NVIC nested vectored interrupt controller
Advanced configurable debug and trace components

	Description
CPU	ARMv7-M, 32bit, 3 stage pipeline
Command set	Thumb-2
BUS	3 AHB-Lite interface Harvard bus architecture : Separate instruction and data interface I-Code : Instruction access D-Code: Data access System: All access to System memory space AMBA (Advanced Microcontroller Bus Architecture) - AHB(Advanced High Performance Bus), - ASP(Advanced System Bus), - APB(Advanced Peripheral Bus)

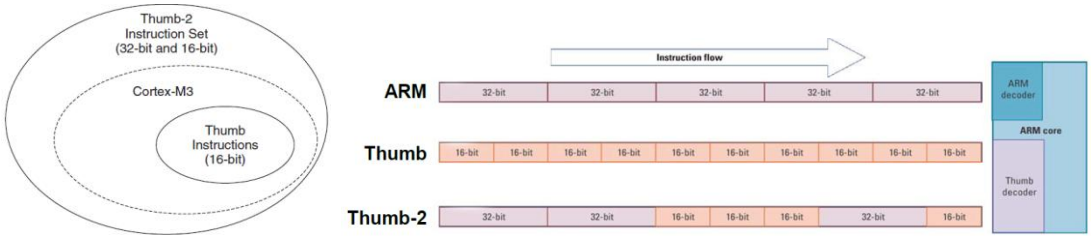
Compare

processor	M3	M4	M7
CPU Core	ARMv7-M	ARMv7E-M	
Bus	32bit		
ISA	RISC		
Memory	Harvard		
Pipeline	3 stage		6
Bus protocol	AMBA		
DMIPS/MHz	1.25	1.25(1.27 with FPU)	2.14
HW Multiplier	64bit		
HW Divider	Yes		
Floating operation		Optional	Optional
Double operation			Optional
Cache			Optional

"STM32 F 1 03 R B T 6" means,

field	Description
STM32	STM32 Family
F	Production Type F:Foundation, G:Mainstream, L:LowPower, H:HighPerformance
1	ARM Cortex Mx. 0:M0 1:M3 2:M3 3:M4 4:M4 7:M7
03	Line (Speed, Peripherals, Silicon Process) information
R	Package F:20pin T:36pin C:48pin R:64,66pin V:100pin Z:144pin I:176pin
B	Flash Size(KB), 4:16, 6:32 8:64 B:128 C:256 D:…
T	Package Type. P:TSOP H:BGA U:QFPN T:LQFP Y:WLCSP
6	Temperature. 6:-40C~85C 7:-40~105C

Instruction Set Architecture (ISA)

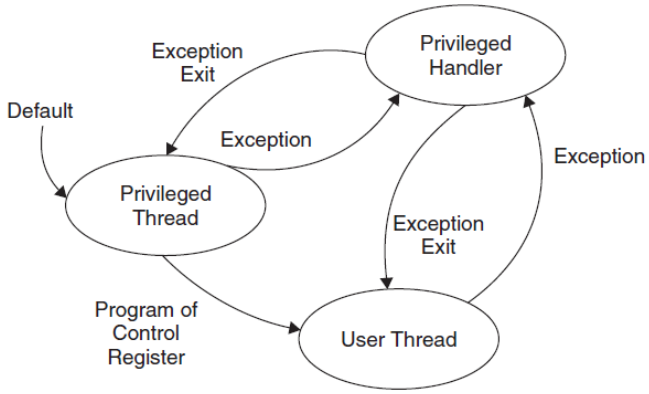


The Thumb-2 instruction set is a superset of the previous 16-bit Thumb instruction set, with additional 16-bit instructions alongside 32-bit instructions.

Privilege Mode

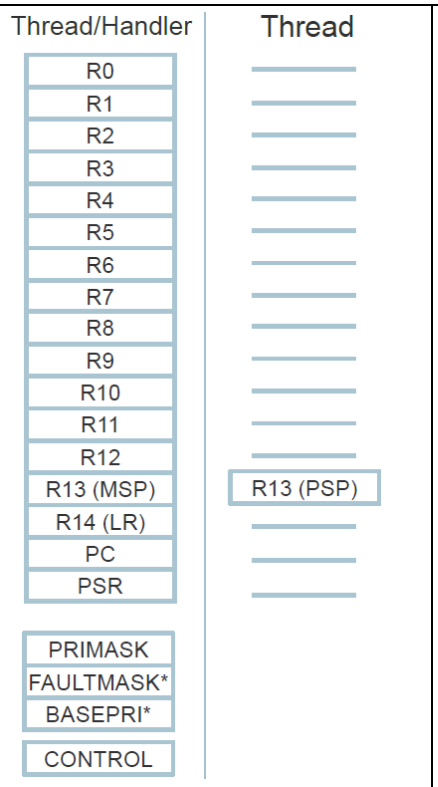
	Privileged	*User*
When running an exception	Handle Mode	
When running main program	Thread Mode	Thread Mode

Operation Modes and Privilege Levels



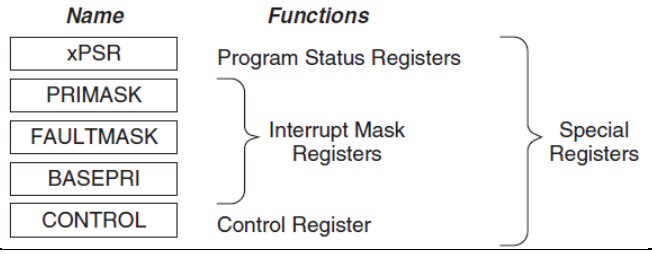
Operation Mode Transitions

1. Register Set



32-bit wide

General Purpose Registers	R0-R7 : access by any instruction R8-R12 : few 16-bit instruction and all 32 bit
3 registers with special	R13: SP Stack Pointer MSP: Main SP, PSP: Process SP R14: LR Link Register Store the return address on subroutine. R15: PC Program Counter
Special purpose registers	PSR : Program Status Register Interrupt Mask Registers PREMASK FAULTMASK BASEPRI CONTROL : Control Register



1.1 Stack Pointer R13

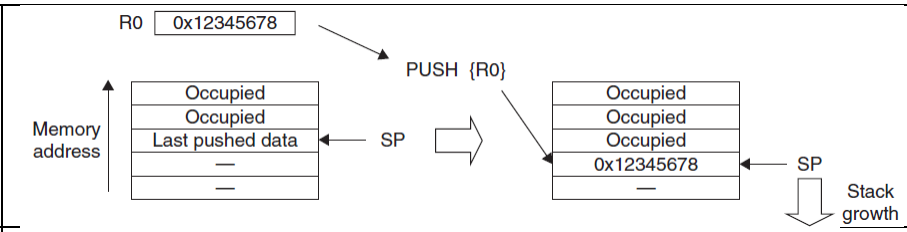
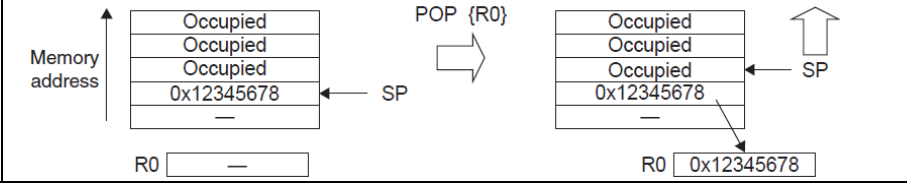
Cortex-M3 processor, there are two stack pointers. This duality allows two separate stack memories to be set up. When using the register name R13, you can only access the current stack pointer; the other one is inaccessible unless you use special instructions MSR and MRS.

	Description	CONTROL[1]
MSP	Main Stack Pointer. This is the default stack pointer; it is used by the OS kernel, exception handlers, and all application codes that require privileged access.	0
PSP	Process Stack Pointer. Used by the base-level application code (when not running an exception handler).	1

You can PUSH or POP multiple registers in one instruction:

```
subroutine_1
    PUSH {R0-R7, R12, R14} ; Save registers
    …                      ; Do your processing
    POP {R0-R7, R12, R14}  ; Restore registers
    BX R14                 ; Return to calling function
```

(Note: You can use MSR/MRS to read/write special registers)

PUSH	
POP	

1.2 Link Register R14

LR is used to store the return program counter when a subroutine or function is called. For example, when you're using the BL (branch and link) instruction:

```
main ; Main program
    …
    BL function1 ; Call function1 using Branch with Link instruction.
                 ; PC _function1 and
                 ; LR _the next instruction in main
    …
function1
    … ; Program code for function 1
    BX LR ; Return
```

1.3 Program Counter R15

When you read PC, it reports the execution instruction plus 4.
Writing to PC will cause a branch.
Instruction address must be half word aligned. Read PC value on LSB is always 0.
On the write, use '1' on LSB to indicate the Thumb state operation.
If LSB was 0, it can be imply trying to switch the ARM state and will be generated a fault exception.

1.4 Access Special Registers

Special registers can only be accessed via MSR and MRS instructions

```
MRS <reg>, <special_reg>; Read special register
MSR <special_reg>, <reg>; write to special register
```

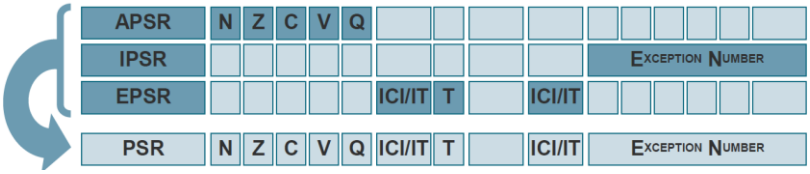
Use MSR/MRS to read/Write the Special Register

command	Description	Example
MSR	Set PSR	MSR PSR,r0
MRS	Read PSR	MRS r1, IPSR

1.5 PSR

Provide ALU flags (zero flag, carry flag), execution status, and current executing interrupt number

APSR	Application Program Status Register Negative, Zero, Carry, OVerflow, Q-Sticky Saturation Flag
IPSR	Interrupt Program Status Register Exception Number –Indicates which exception processor is handling
EPSR	Execution Program Status Register ICI/IT –Interrupt Continuable Instruction, IF-THEN instruction status Thumb –Always 1



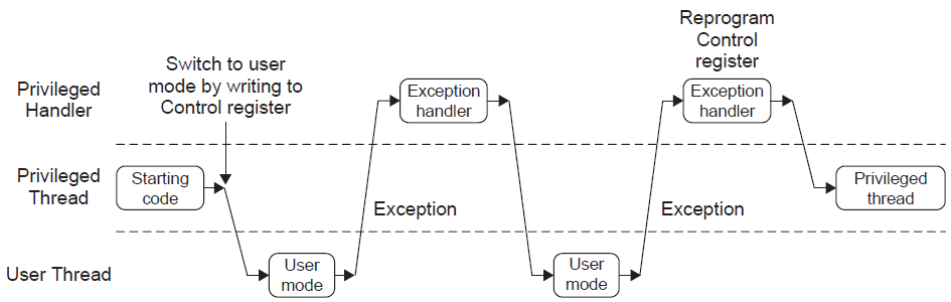
1.6 Interrupt Mask Registers

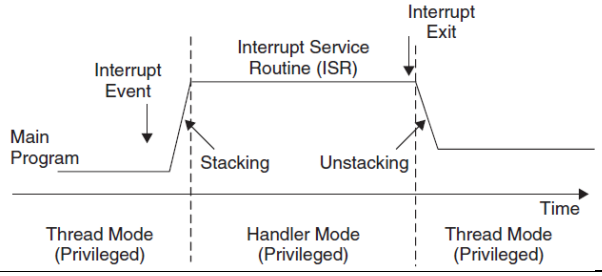
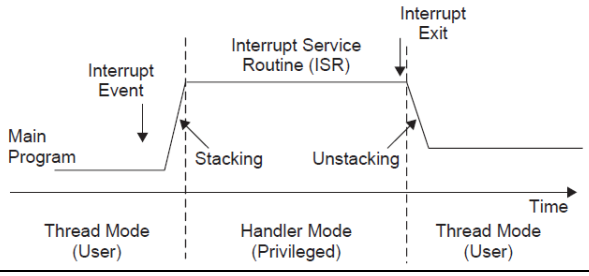
	Description
PRIMASK	A 1-bit register. When this is set, it allows NMI and the hard fault exception; all other interrupts and exceptions are masked. The default value is 0, which means that no masking is set.
FAULTMASK	A 1-bit register. When this is set, it allows only the NMI, and all interrupts and fault handling exceptions are disabled. The default value is 0, which means that no masking is set.
BASEPRI	A register of up to 9 bits (depending on the bit width implemented for priority level). It defines the masking priority level. When this is set, it disables all interrupts of the same or lower level (larger priority value). Higher-priority interrupts can still be allowed. If this is set to 0, the masking function is disabled (this is the default).

1.7 CONTROL

It is used to define the privilege level and the stack pointer selection.

Bit	Function
CONTROL[1]	Stack status: 1 : Alternate stack is used 0 : Default stack (MSP) is used If it is in the Thread or base level, the alternate stack is the PSP. There is no alternate stack for handler mode, so this bit must be zero when the processor is in handler mode.
CONTROL[0]	0 : Privileged in Thread mode 1 : User state in Thread mode If in handler mode (not Thread mode), the processor operates in privileged mode.



Switching Processor Mode at Interrupt	Switching Processor Mode and Privilege Level at Interrupt
	

CONTROL [1]=0: Both Thread Level and Handler Use Main Stack (MSP)

CONTROL [1]=1: Thread Level Uses Process Stack (PSP) and Handler Uses Main Stack (MSP)

2 Components

2.1 Nested Vectored Interrupt Controller (NVIC)

It is closely coupled to the processor core.

Features	Description
Nested interrupt support	When an interrupt occurs, the NVIC compares the priority of this interrupt to the current running priority level. If the priority of the new interrupt is higher than the current level, the interrupt handler of the new interrupt will override the current running task.
Vectored interrupt support	the vector table have location address for the interrupt service routine (ISR).
Dynamic priority changes support	Priority levels of interrupts can be changed by software during run time.
Reduction of interrupt latency	tail chaining interrupts, late arrival interrupts
Interrupt masking	BASEPRI, PRIMASK, and FAULTMASK

2.2 Memory Protection Unit (MPU)

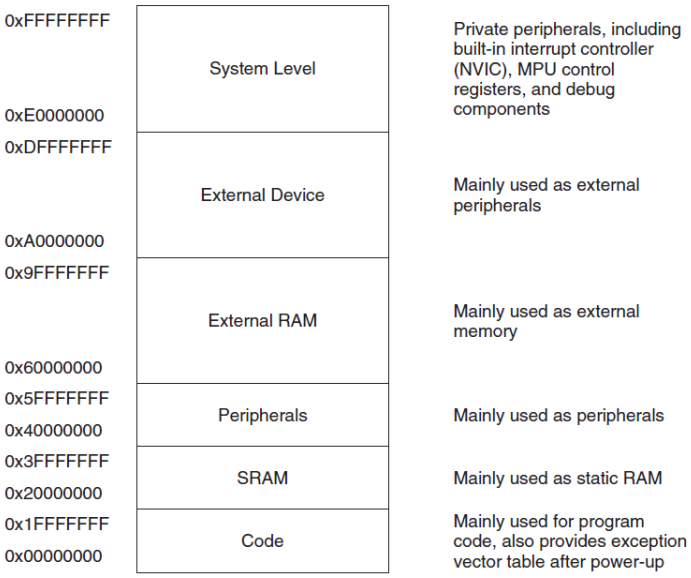
This unit allows access rules to be set up for privileged access and user program access. When an access rule is violated, a fault exception is generated, and the fault exception handler will be able to analyze the problem and correct it if possible.

It provides the privilege mode on OS, protects read-only memory, isolate memory regions between different tasks.

2.3 The Bus Interface

	Description
Code memory bus	physically consist of two buses, one called *I-Code* and another called *D-Code*.
system bus	is used to access memory and peripherals. Access to the SRAM, peripherals, external RAM, external devices, and part of the system-level memory regions.
private peripheral bus	access to a part of the system-level memory dedicated to private peripherals such as debugging components.

3. Memory Map



3.1 Vector Tables

Exception Type	Address Offset	Exception Vector
18–255	0x48–0x3FF	IRQ #2–239
17	0x44	IRQ #1
16	0x40	IRQ #0
15	0x3C	SYSTICK
14	0x38	PendSV
13	0x34	Reserved
12	0x30	Debug Monitor
11	0x2C	SVC
7–10	0x1C–0x28	Reserved
6	0x18	Usage fault
5	0x14	Bus fault
4	0x10	MemManage fault
3	0x0C	Hard fault
2	0x08	NMI
1	0x04	Reset
0	0x00	Starting value of the MSP

3.2 Exception and Interrupts

Exception Number	Exception Type	Priority (Default to 0 if Programmable)	Description
0	NA	NA	exception running
1	Reset	-3 (Highest)	Reset

2	NMI	-2	Non-mask-able interrupt. (external NMI input)
3	Hard fault	-1	All fault conditions, if the corresponding fault handler is not enabled
4	MemManage fault	Programmable	Memory management fault; MPU violation or access to illegal locations
5	Bus fault	Programmable	Bus error (Prefetch Abort or Data Abort)
6	Usage fault	Programmable	Exceptions due to program error
7-10	Reserved	NA	Reserved
11	SVCall	Programmable	System service call
12	Debug monitor	Programmable	Debug monitor (break points, watchpoints, or external debug request)
13	Reserved	NA	Reserved
14	PendSV	Programmable	Pendable request for system device
15	SYSTICK	Programmable	System tick timer
16	IRQ #0	Programmable	External interrupt #0
..	..	..	..
255	IRQ #239	Programmable	External interrupt #239

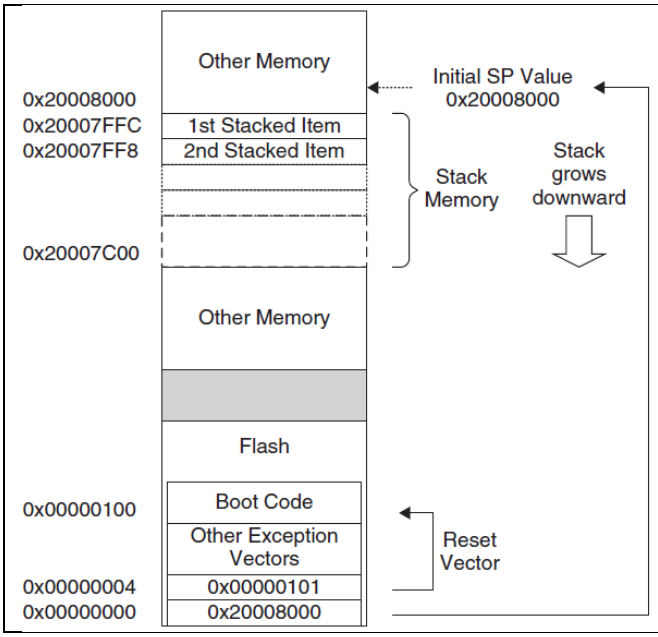
3.3 Reset Sequence

After the processor exits reset, it will read two words from memory:

• Address 0x00000000: Starting value of R13 (the stack pointer)

• Address 0x00000004: Reset vector (the starting address of program execution; LSB should be set to 1 to indicate Thumb state)

At first set the initial stack pointer value, and then, after the reset vector is fetched, the Cortex-M3 can then start to execute the program from the reset vector address and begin normal operations.



The vector table starts after the initial SP value.

The vector table, which contains vector address values. (The vector table can be relocated to another location later, during program execution.)

On the example, SP value was 0x20008000, and stack grows downward.

The Reset Vector has 0x00000101 and Boot code starts from 0x00000100. The LSB 1 means the Thumb2 command.

4 Instruction Sets

4.1 Basic

4.1.1 Basic Syntax

```
label
    opcode operand1, operand2,... ; Comments
```

label, opcode, operand and comment.

immediate data

```
    MOV R0, #0x12 ; Set R0 = 0x12 (hexadecimal)
    MOV R1, #'A' ; Set R1 = ASCII character A
```

constants using EQU.

```
NVIC_IRQ_SETEN0 EQU 0xE000E100
NVIC_IRQ0_ENABLE EQU 0x1
    ...
    LDR R0,_NVIC_IRQ_SETEN0  ; LDR here is a pseudo instruction that
                             ; convert to a PC relative load by assembler.
    MOV R1,#NVIC_IRQ0_ENABLE ; Move immediate data to register
    STR R1, [R0]             ; Enable IRQ 0 by writing R1 to address in R0
```

DCI can be used to code an instruction

```
  DCI 0xBE00 ; Breakpoint (BKPT 0), a 16-bit instruction
```

DCB (for byte size constant values such as characters) and DCD (for word size constant values) to define binary data in your code:

```
    LDR R3,=MY_NUMBER ; Get the memory address value of MY_NUMBER
    LDR R4,[R3]       ; Get the value code 0x12345678 in R4
    ...
    LDR R0,=HELLO_TXT ; Get the starting memory address of HELLO_TXT
    BL PrintText      ; Call a function called PrintText to display string
    ...
MY_NUMBER
    DCD 0x12345678
HELLO_TXT
    DCB "Hello\n",0 ; null terminated string
```

4.1.2 Suffix

Suffix	Description
S	Update APSR (flags); for example: ADDS R0, R1 ; this will update APSR
EQ , NE, LT, GT, and so on	Conditional execution; EQ Equal, NE Not Equal, LT Less Than, GT Greater Than, etc. For example: BEQ <Label> ; Branch if equal

Note: APSR means Application Program Status Register

4.1.3 Unified Assembler Language

ADD R0, R1	R0 = R0 + R1, using Traditional Thumb syntax
ADD R0, R0, R1	Equivalent instruction using UAL syntax (Unified Assembler Language)

AND R0, R1	Traditional Thumb syntax
ANDS R0, R0, R1	Equivalent UAL syntax (S suffix is added)

For example, "R0 = R0 + 1" can be implemented as a 16-bit Thumb instruction or a 32-bit Thumb-2 instruction.

```
ADDS R0, #1     ; Use 16-bit Thumb instruction by default for smaller size
ADDS.N R0, #1  ; Use 16-bit Thumb instruction (N=Narrow)
ADDS.W R0, #1 ; Use 32-bit Thumb-2 instruction (W=wide)
```

32-bit Thumb-2 instructions can be half word aligned.

```
0x1000 : LDR r0,[r1]   ; a 16-bit instructions (occupy 0x1000-0x1001)
0x1002 : RBIT.W r0    ; a 32-bit Thumb-2 instruction (occupy 0x1002-0x1005)
0x1006 :
```

4.2 Instruction List

16-Bit Data Processing Instructions 16-Bit Branch 16-Bit Load and Store Other 16-Bit	32-Bit Data Processing Instructions 32-Bit Branch 32-Bit Load and Store	Unsupported Instructions BLX label SETEND Unsupported Coprocessor Instructions Unsupported Change Process State (CPS) Instructions Unsupported Hint Instructions

4.3 Instruction Descriptions

4.3.1 Moving Data

MOV : Move
MVN : generate the negative value of the original data
LDR: Load
STR: Store
LDM: Load Multiple
STM: Store Multiple

Exclamation mark (!) in the instruction specifies whether the register *Rd* should be updated after the instruction is completed.

```
    STMIA.W R8!, {R0-R3} ; R8 changed to 0x8010 after store
                         ; (increment by 4 words)
    STMIA.W R8 , {R0-R3} ; R8 unchanged after store
```

pre-indexing and post-indexing.
For preindexing, the register holding the memory address is adjusted. The memory transfer then takes place with the updated address. For example:

```
    LDR.W R0,[R1, #offset]! ; Read memory[R1+offset],
                            ; with R1 update to R1+offset
```

Post-indexing memory access instructions carry out the memory transfer using the base address specifi ed by the register and then update the address register afterward. For example:

```
    LDR.W R0,[R1], #offset ; Read memory[R1], with R1
                           ; updated to R1+offset
```

Push & Pop

```
    PUSH {R0, R4-R7, R9} ; Push R0, R4, R5, R6, R7, R9 into stack memory
    POP {R2,R3} ; Pop R2 and R3 from stack
```

```
    PUSH {R0-R3, LR} ; Save register contents at beginning of subroutine
    .... ; Processing
    POP {R0-R3, PC} ; restore registers and return
```

For Special Registers

```
    MRS R0, PSR ; Read Processor status word into R0
    MSR CONTROL, R1 ; Write value of R1 into control register
```

Unless you're accessing the APSR, you can use MSR or MRS to access other special registers only in privileged mode.

Move immediate data into a register

```
    MOV R0, #0x12 ; Set R0 to 0x12. 8bits or less
```

```
    MOVW.W R0,#0x789A ; Set R0 to 0x789A. over 8bits
```

```
    MOVW.W R0,#0x789A ; Set R0 lower half to 0x789A
    MOVT.W R0,#0x3456 ; Set R0 upper half to 0x3456. Now R0=0x3456789A
```

```
    LDR R0, =0x3456789A ; or use pseudo instruction provided in ARM assembler
; it gives better readability and the assembler might be able to reduce the
memory being used.
```

4.3.2 LDR and ADR Pseudo Instructions

if the address is a program address value, it will automatically set the LSB to 1.

```
    LDR R0, =address1 ; R0 set to 0x4001
    ...
address1
0x4000: MOV R0, R1 ; address1 contains program code
...
```

If *address1* is a data address, LSB will not be changed. For example:

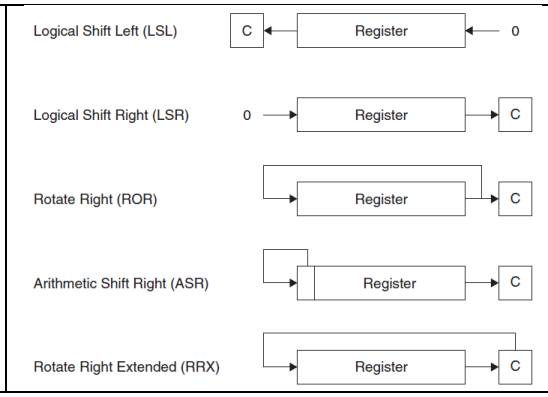
```
    LDR R0, =address1 ; R0 set to 0x4000
    ...
address1
0x4000: DCD 0x0 ; address1 contains data
```

you can load the address value of a program code into a register without setting the LSB automatically. For example:

```
    ADR R0, address1
    ...
address1
0x4000: MOV R0, R1 ; address1 contains program code
```

Note that there is no equal sign (=) in the ADR statement.

4.3.3 Processing Data

Command	Description
ADD	Add
SUB	Subtract
MUL	Multiply
UDIV/SDIV	Unsigned and signed divide
SMULL, SMLAR UMULL, UMLAR	32-bit multiply instructions and multiply accumulate instructions that give 64-bit results.
AND ORR BIC ORN EOR	Bitwise AND logical operation Bitwise OR Bit clear Bitwise OR NOT Bitwise Exclusive OR
ASR LSL LSR ROR RRX	Arithmetic shift right Logic shift left Logic shift right Rotate right Rotate right extended 
	No Rotate Left. The rotate left operation can be replaced by a rotate right operation with a different rotate offset. For example, a rotate left by 4-bit operation can be written as a rotate right by 28-bit instruction, which gives the same result and takes the same amount of time to execute.
SXTB.W SXTH.W	Sign extend byte data into word Sign extend half word data into word
REV.W REV16.W REVSH.W	Reverse bytes in word Reverse bytes in each half word Reverse bytes in bottom half word and sign extend the result
 BFC.W BFI.W CLZ.W RBIT.W SBFX.W UBFX.W	Bit Field Processing and Manipulation Instructions Clear bit field within a register Insert bit field to a register Count leading zero Reverse bit order in register Copy bit field from source and sign extend it Copy bit field from source register

4.3.4 Call and Unconditional Branch

Branch

```
    B label ; Branch to a labeled address
    BX reg ; Branch to an address specifi ed by a register
```

Branch with LR

```
    BL label ; Branch to a labeled address and save return address in LR
    BLX reg  ; Branch to an address specified by a register and
             ; save return address in LR.
```

combine with POP

```
main
    ...
    BL functionA
    ...
functionA
    PUSH {LR} ; Save LR content to stack
    ...
    BL functionB
    ...
    POP {PC} ; Use stacked LR content to return to main
functionB
    PUSH {LR}
    ...
    POP {PC} ; Use stacked LR content to return to functionA
```

4.3.5 Decision and Conditional Branch

Flag bits in APSR (Application Program Status Register) That Can Be Used for Conditional Branches.

Flag	PSR Bit	Description
N	31	Negative flag (last operation result is a negative value) This flag is set when the result of an instruction has a negative value (bit 31 is 1).
Z	30	Zero (last operation result returns a zero value) This flag is set when the result of an instruction has a zero value or when a comparison of two data returns an equal result.
C	29	Carry (last operation returns a carry out or borrow) This flag is for unsigned data processing—for example, in add (ADD) it is set when an overflow occurs; in subtract (SUB) it is set when a borrow did not occur (borrow is the invert of carry).
V	28	Overflow (last operation results in an overflow) This flag is for signed data processing; for example, in an add (ADD), when two positive values added together produce a negative value, or when two negative values added together produce a positive value.

Conditions for Branches or Other Conditional Operations

Symbol	Condition	Flag
EQ	Equal	Z set
NE	Not equal	Z clear
CS/HS	Carry set/unsigned higher or same	C set
CC/LO	Carry clear/unsigned lower	C clear
MI	Minus/negative	N set
PL	Plus/positive or zero	N clear
VS	Overflow	V set
VC	No overflow	V clear
HI	Unsigned higher	C set and Z clear
LS	Unsigned lower or same	C clear or Z set
GE	Signed greater than or equal	N set or V set, or N clear and V clear (N == V)
LT	Signed less than	N set and V clear, or N clear and V set (N != V)
GT	Signed greater than	Z clear, and either N set and V set, or N clear and V clear (Z == 0, N == V)
LE	Signed less than or equal	Z set, or N set and V clear, or N clear and V set (Z == 1 or N != V)
AL	Always (unconditional)	–

4.3.6 Combined Compare and Conditional Branch

C	ASM

`i = 5;`
`while (i != 0 ){`
`  func1();`
`  i--;`
`}`
	`    MOV R0, #5          ; Set loop counter`
	`loop1 CBZ R0, loop1exit ; if loop counter = 0`
	`                        ; then exit the loop`
	`    BL func1            ; call a function`
	`    SUB R0, #1          ; loop counter decrement`
	`    B loop1             ; next loop`
	`loop1exit`

4.3.7 Conditional Branches Using IT Instructions

The IT (IF-THEN) block. It can provide a maximum of four conditionally executed instructions.

```
IT<x><y><z> <cond> ; IT instruction (<x>, <y>, <z> can be T or E)
instr1<cond> <operands> ; 1st instruction (<cond> must be same as IT)
instr2<cond or not cond> <operands> ; 2nd instruction (can be <cond> or <!cond>
instr3<cond or not cond> <operands> ; 3rd instruction (can be <cond> or <!cond>
instr4<cond or not cond> <operands> ; 4th instruction (can be <cond> or <!cond>
```

C	ASM

`if (R1<R2) then`
`  R2=R2-R1`
`  R2=R2/2`
`else`
`  R1=R1-R2`
`  R1=R1/2`
	`CMP R1, R2 ; If R1 < R2 (less then)`
	`ITTEE LT ; then execute instruction 1 and 2`
	`         ; (indicated by T)`
	`         ; else execute instruction 3 and 4`
	`         ; (indicated by E)`
	`SUBLT.W R2,R1 ; 1st instruction`
	`LSRLT.W R2,#1 ; 2nd instruction`
	`SUBGE.W R1,R2 ; 3rd instruction (notice the GE is`
	`              ; opposite of LT)`

	`LSRGE.W R1,#1 ; 4th instruction`

4.3.8 Instruction Barrier and Memory Barrier Instructions

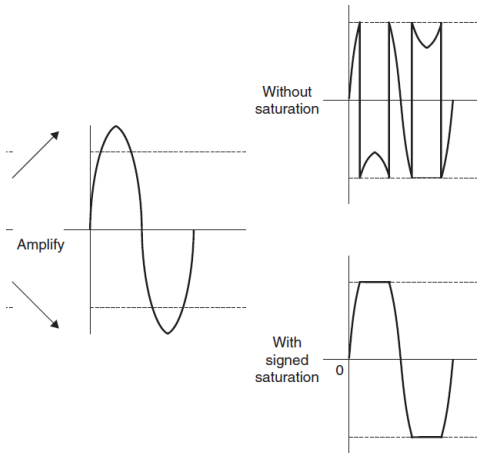
In some cases, if memory barrier instructions are not used, race conditions could occur.

Instruction	Description
DMB	Data Memory Barrier; ensures that all memory accesses are completed before new memory access is committed
DSB	Data Synchronization Barrier; ensures that all memory accesses are completed before next instruction is executed
ISB	Instruction Synchronization Barrier; flushes the pipeline and ensures that all previous instructions are completed before executing new instructions

if the memory map can be switched by a hardware register, after writing to the memory switching register you should use the **DSB** instruction. Otherwise, if the write to the memory switching register is buffered and takes a few cycles to complete, and the next instruction accesses the switched memory region immediately, the access could be using the old memory map.

4.3.9 Saturation Operations

SSAT and USAT



Instruction	Description
SSAT.W <Rd>, #<immed>, <Rn>, {,<shift>}	Saturation for signed value
USAT.W <Rd>, #<immed>, <Rn>, {,<shift>}	Saturation for a signed value into an unsigned value

The Q flag is set if saturation takes place in the operation, and it can be cleared by writing to the APSR.

4.4 Several Useful Instructions in the Cortex-M3

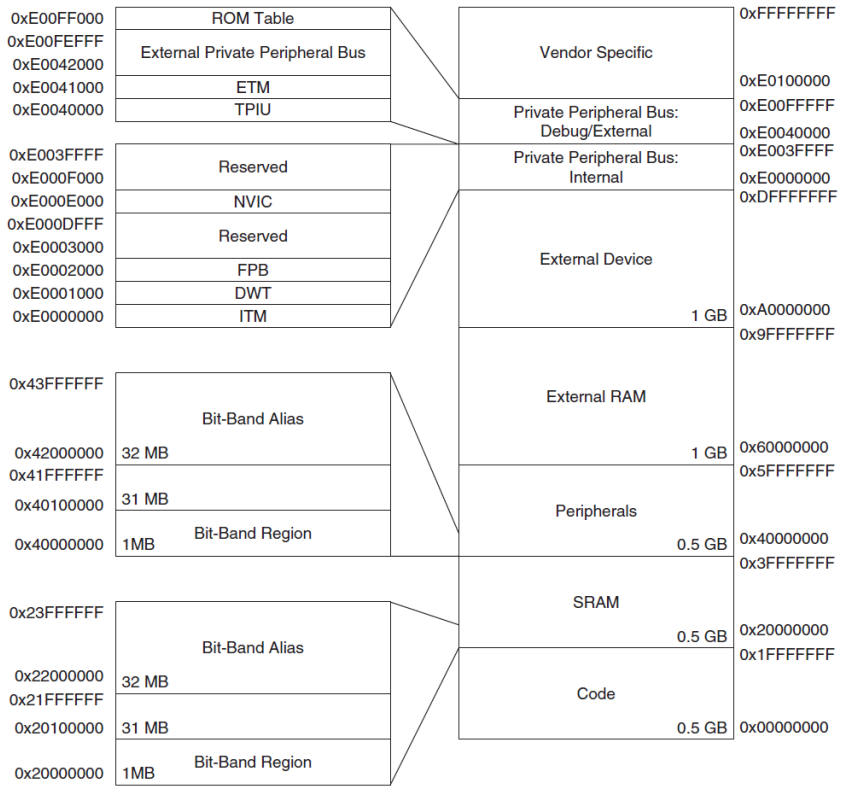
Command	Description

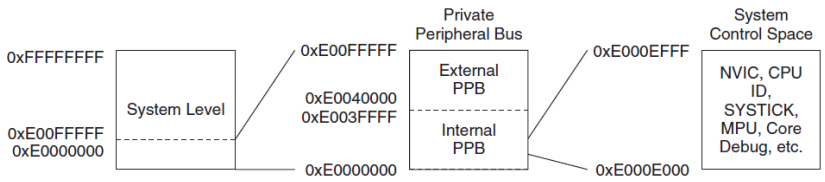
MSR and MRS	These two instructions provide access to the special registers
IF-THEN (IT)	The IF-THEN (IT) instructions allow up to four succeeding instructions (called an *IT block*) to be conditionally executed.
CBZ and CBNZ	The compare and then branch if zero/nonzero instructions are useful for looping (for example, the WHILE loop in C).
SDIV and UDIV	The syntax for signed and Unsigned divide instructions. The result is Rd _ Rn/Rm
REV, REVH and REVSH	REV reverses the byte order in a data word, and REVH reverses the byte order inside a half word.
RBIT	The RBIT instruction reverses the bit order in a data word.
SXTB, SXTH, UXTB, UXTH	The four instructions SXTB, SXTH, UXTB, and UXTH are used to extend a byte or half word data into a word.
BFC and BFI	BFC (Bit Field Clear) clears any number of adjacent bits in any position of a register.
UBFX and SBFX	UBFX and SBFX are the Unsigned and Signed Bit Field Extract instructions.
LDRD, STRD	The two instructions LDRD and STRD transfer two words of data from or into two registers.
TBB and TBH	TBB (Table Branch Byte) and TBH (Table Branch Halfword) are for implementing branch tables.

5. Memory Systems

it has a predefined memory map.
the bit-band support. This provides atomic operations to bit data in memory or peripherals.
unaligned transfers and exclusive accesses

5.1 Predefined Memory Map





5.2 Memory Access Attributes

Bufferable, Cacheable, Executable, Shareable

If MPU, can be overridden.

		attribute	cache	
Code	0.5G 0x00000000–0x1FFFFFFF	executable	WT	You can put data memory in this region as well. Write is buffered.
SRAM	0.5G (0x20000000–0x3FFFFFFF	Write is buffered executable	WB-WA	intended for on-chip RAM
Peripheral	0x40000000–0x5FFFFFFF	No exec.	noncacheable	intended for peripherals.
External RAM	(0x60000000–0x7FFFFFFF	executable	WB-WA	intended for either on-chip or off-chip memory.
External RAM	0x80000000–0x9FFFFFFF	executable	WT	same
External Device	0xA0000000–0xBFFFFFFF	No exec.	noncacheable	external devices and/or shared memory that needs ordering/nonbuffered accesses.
Eextrnal devices	0xC0000000–0xDFFFFFFF	No exec.	noncacheable	same
System region	0xE0000000–0xFFFFFFFF	nonexecutable.	noncacheable	private peripherals and vendor-specific devices. For the private peripheral bus memory range, the accesses are strongly ordered (noncacheable, nonbufferable). For the vendor-specific memory region, the accesses are bufferable and noncacheable.

Cache

	Description
WT	Write-Through In a Write-Through scheme, every time the CPU performs a write operation to the cache, the data is **simultaneously** written to the main memory. **- Mechanism:** The cache and RAM are updated at the same time. **- Pros:** Data consistency (coherency) is always maintained between cache and RAM. If a system crash occurs, the RAM has the most recent data. **- Cons:** It is slower because the CPU must wait for the slower main memory write to complete before moving to the next instruction.
WB	Write-Back In a Write-Back scheme, the CPU writes data only to the cache. The main memory is updated only when that specific cache line is **evicted** (replaced by new data). **- Mechanism:** Uses a "Dirty Bit" to mark cache lines that have been modified but not yet saved to RAM. **- Pros:** Significantly faster performance for write-intensive applications because it minimizes the number of times the system accesses the slower main memory. **- Cons:** More complex to implement. There is a risk of data loss if the system fails before the "dirty" data is written back to RAM.

Handling Write Misses

When the CPU tries to write to a location that isn't in the cache, it uses one of two primary strategies:

	Description
WA	Write-Allocate This is almost always paired with **Write-Back**. **- Process:** The relevant block is loaded from main memory into the cache, and then the write is

	performed in the cache. - **Logic:** It assumes that if you wrote to a location once, you will likely write to it (or near it) again soon (temporal/spatial locality).
NWA	No-Write-Allocate This is often paired with **Write-Through**. - **Process:** The data is written directly to the main memory, and the cache is not updated. - **Logic:** This avoids "polluting" the cache with data that might not be read again immediately.

5.3 Default Access Permissions

Memory Region	Address	Access in User Program
Vendor specific	0xE0100000–0xFFFFFFFF	Full access
ROM Table	0xE00FF000–0xE00FFFFF	Blocked; user access results in bus fault
External PPB	0xE0042000–0xE00FEFFF	Blocked; user access results in bus fault
ETM	0xE0041000–0xE0041FFF	Blocked; user access results in bus fault
TPIU	0xE0040000–0xE0040FFF	Blocked; user access results in bus fault
Internal PPB	0xE000F000–0xE003FFFF	Blocked; user access results in bus fault
NVIC	0xE000E000–0xE000EFFF	Blocked; user access results in bus fault, except Software Trigger Interrupt Register that can be programmed to allow user accesses
FPB	0xE0002000–0xE0003FFF	Blocked; user access results in bus fault
DWT	0xE0001000–0xE0001FFF	Blocked; user access results in bus fault
ITM	0xE0000000–0xE0000FFF	Read allowed; write ignored except for stimulus ports with user access enabled
External Device	0xA0000000–0xDFFFFFFF	Full access
External RAM	0x60000000–0x9FFFFFFF	Full access
Peripheral	0x40000000–0x5FFFFFFF	Full access
SRAM	0x20000000–0x3FFFFFFF	Full access
Code	0x00000000–0x1FFFFFFF	Full access

5.4 Bit-Band

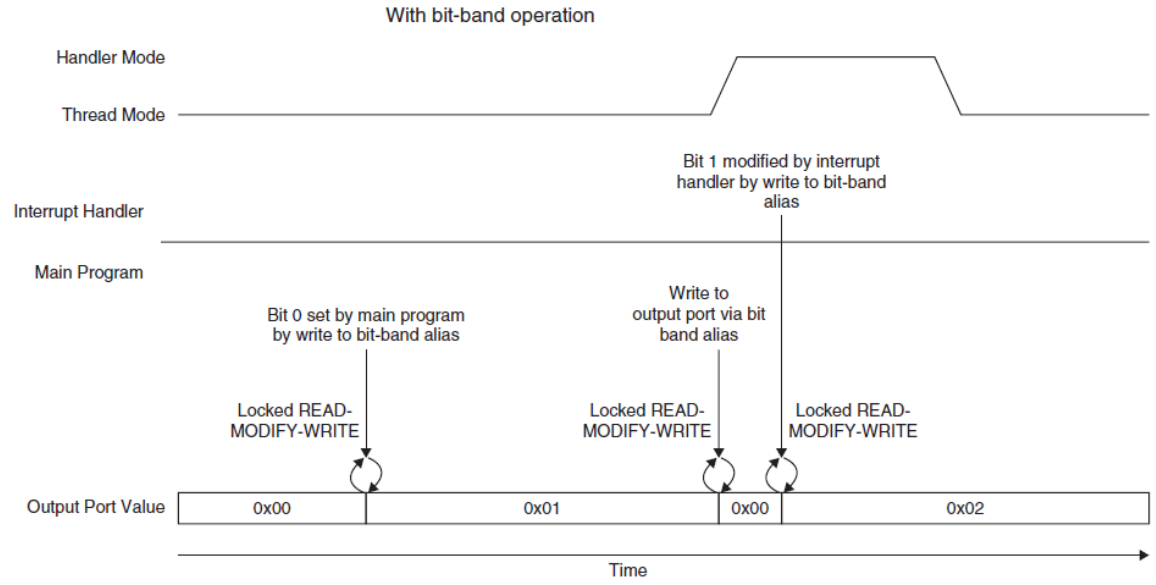
Atomic READ_MODIFY_WRITE operation.
When the bit-band alias address is accessed, the address is remapped into a bit-band address.

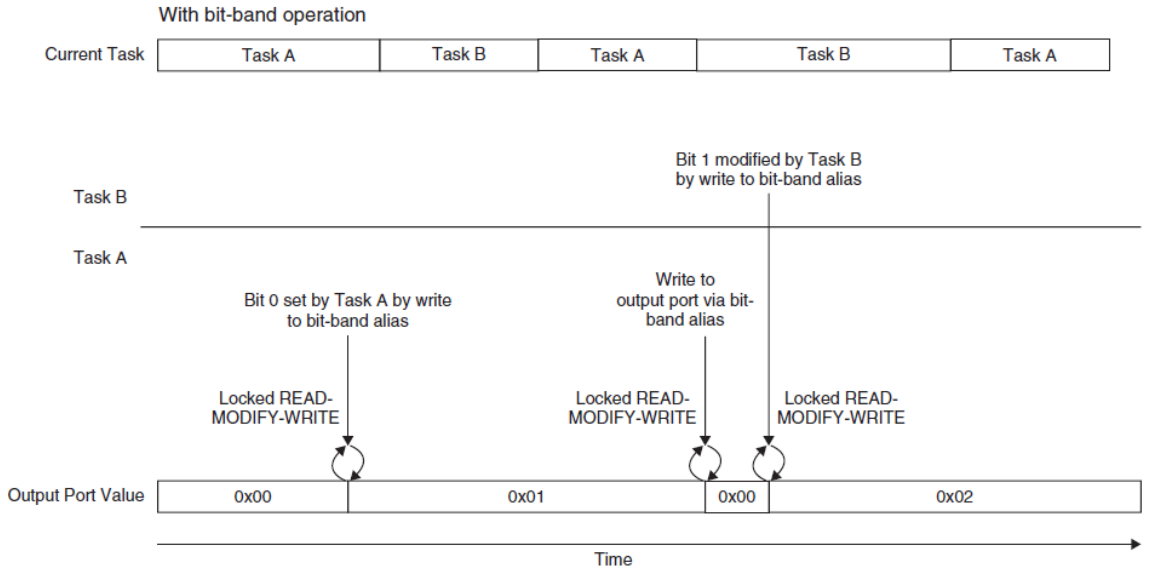
Bit-Band address. 1MB	Bit-Band alas address. 32MB
0x20000000 SRAM	0x22000000 Note: (Bit-Bank << 2 + 0x22000000)
0x40000000 Peripheral	0x42000000

Unaligned access will be unpredictable!

```
// Convert bit band address and bit number into bit band alias address
#define BITBAND(addr,bitnum) \
   ((addr & 0xF0000000)+0x2000000+((addr & 0xFFFFF)<<5)+(bitnum <<2))
// Convert the address as a pointer
#define MEM_ADDR(addr) *((volatile unsigned long *) (addr))
```

```
#define DEVICE_REG0 0x40000000
#define BITBAND(addr,bitnum) ((addr & 0xF0000000)+0x02000000+((addr &
0xFFFFF)<<5)_(bitnum<<2))
#define MEM_ADDR(addr) *((volatile unsigned long *) (addr))
…
```





C example

```
#define DEVICE_REG0 ((volatile unsigned long *) (0x40000000))
#define DEVICE_REG0_BIT0 ((volatile unsigned long *) (0x42000000))
#define DEVICE_REG0_BIT1 ((volatile unsigned long *) (0x42000004))
...
*DEVICE_REG0 = 0xAB; // Accessing the hardware register by normal address
...
```

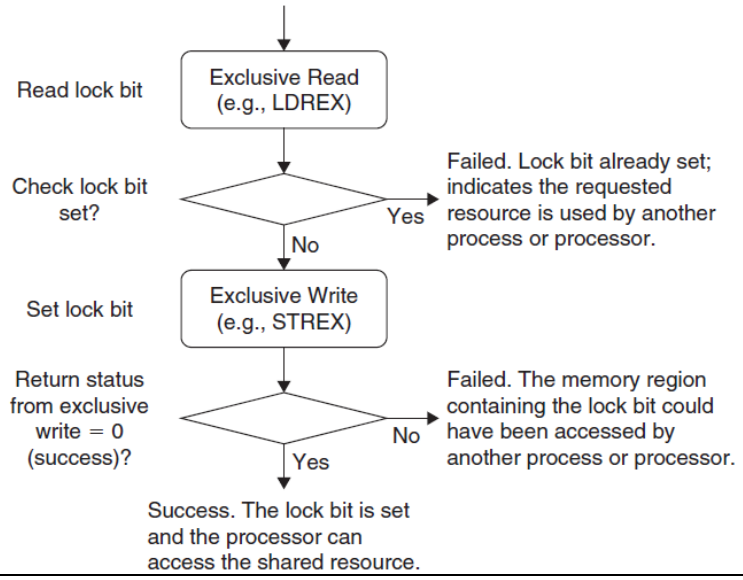
```
*DEVICE_REG0 = *DEVICE_REG0 | 0x2; // Setting bit 1 without using bitband feature
...
*DEVICE_REG0_BIT1 = 0x1; // Setting bit 1 using bitband feature
                         // via the bit band alias address
```

Unaligned Transfer

Data memory accesses can be defined as aligned or unaligned.
when an unaligned transfer takes place, it is broken into separate transfers and as a result it takes more clock cycles for a single data access
It is also possible to set up the NVIC so that an exception is triggered when an unaligned transfer takes place. UNALIGN_TRP (Unaligned Trap) bit in the Confi guration Control Register in the NVIC (0xE000ED14).

Exclusive Access

No SWP instruction.
In newer ARM processors, the read/write access can be carried out on separated buses.
Therefore, the locked transfers are replaced by exclusive accesses.
LDREX / STREX



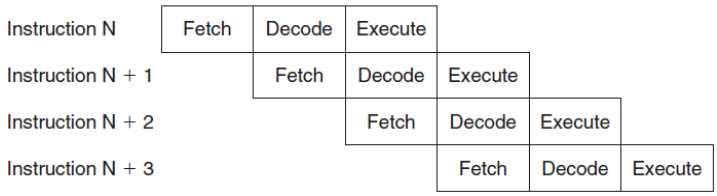

```
LDREX <Rxf>, [Rn, #offset]
STREX <Rd>, <Rxf> ,[Rn, #offset]
```

Endian Mode

Instruction fetches are always in little endian

6. Implementation Overview

Pipeline

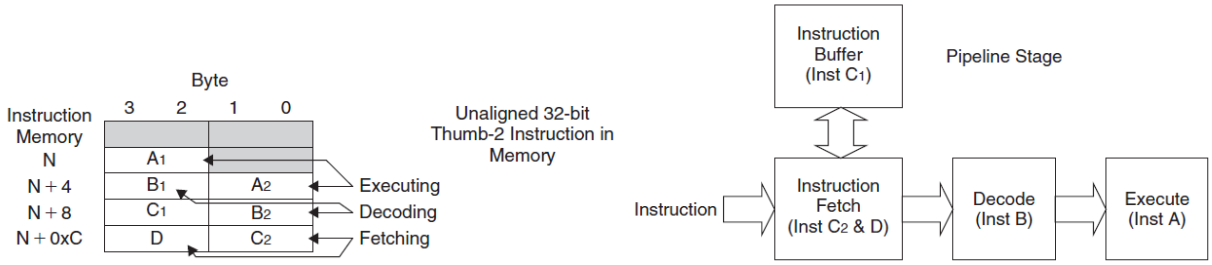


stall

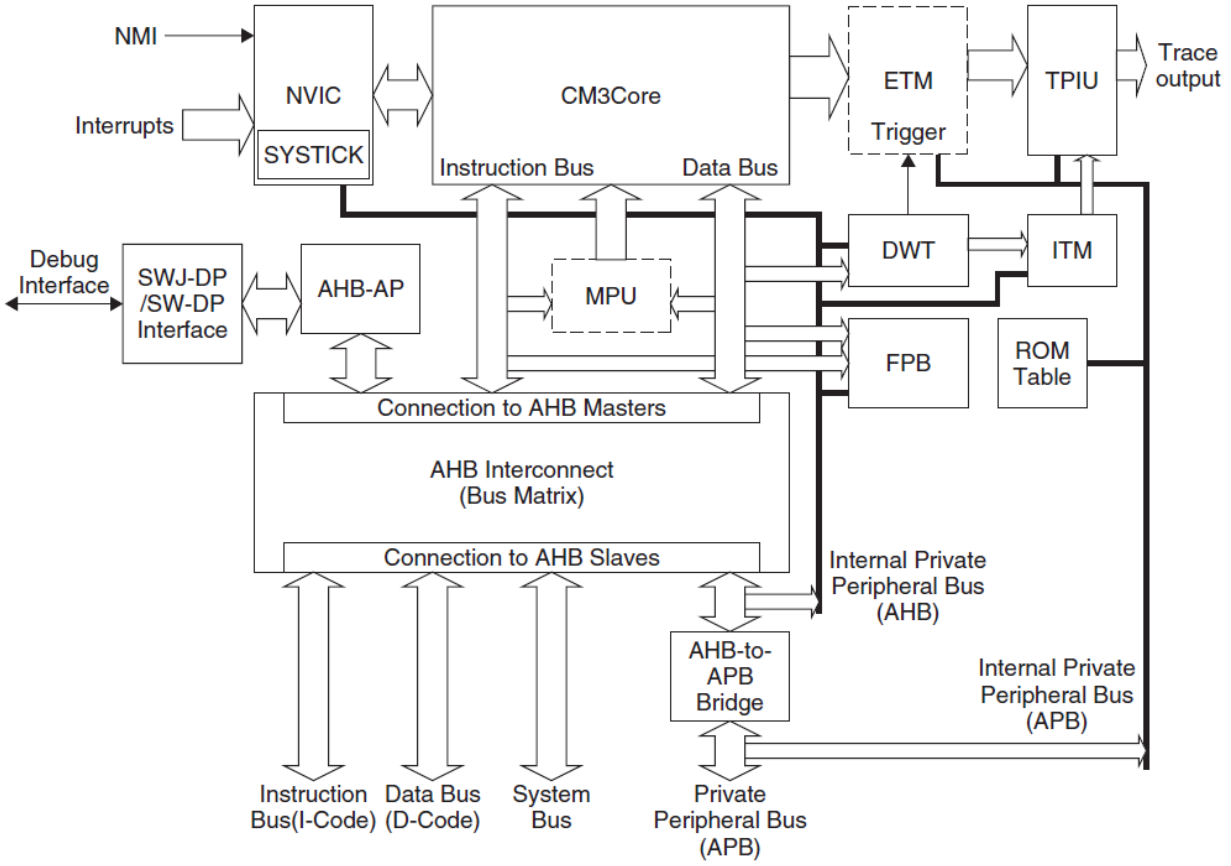
pre-fetch

This buffer allows additional instructions to be queued before they are needed.

This buffer prevents the pipeline being stalled when the instruction sequence contains 32-bit Thumb-2 instructions that are not word aligned.

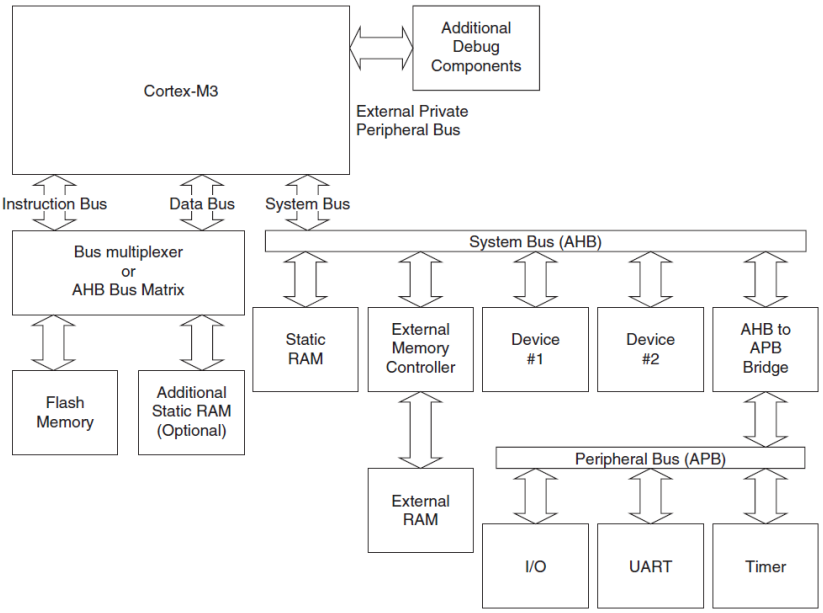


6.1 Detail Block Diagram



Name	Description
Core	registers + ALU + data path, bus interface.
NVIC	build-in nested interrupt controller.
SYSTICK	A simple timer that can be used by the operating system. It is a basic countdown timer, even when the system is in sleep mode.
MPU	option. Protect read-only, privilege data
BusMatrix	Internal AHB interconnection
AHB to APB	Bus bridge to convert AHB to APB
SW-DP/SWJ-DP interface	Serial Wire/Serial Wire JTAG debug port (DP) interface; debug interface connection implemented using either Serial Wire Protocol or traditional JTAG Protocol (for SWJ-DP)
AHB-AP	AHB Access Port; converts commands from Serial Wire/SWJ interface into AHB transfers
ETM	Embedded Trace Macrocell; a module to handle instruction trace for debug (optional)
DWT	Data Watchpoint and Trace unit; a module to handle the data watchpoint function for debug
ITM	Instrumentation Trace Macrocell
TPIU	Trace Port Interface Unit; an interface block to send debug data to external trace capture hardware
FPB	Flash Patch and Breakpoint
ROM table	A small lookup table that stores configuration information

6.1 Bus Interface



	Description
I-Code Bus	32-bit bus based on the AHB-Lite bus protocol for instruction fetches. Instruction fetches are performed in word size, even for Thumb instructions. Therefore, during execution, the CPU core could fetch up to two Thumb instructions at a time.
D-Code Bus	32-bit bus based on the AHB-Lite bus protocol; it is used for data access. Although the Cortex-M3 processor supports unaligned transfers, the bus interface on the processor core converts the unaligned transfers into aligned transfers for you.
System Bus	32-bit bus based on the AHB-Lite bus protocol. all transfers are aligned.
External Private Peripheral Bus	External PPB. 32-bit bus based on the APB bus (Advance Peripheral Bus) protocol. Transfers on this bus are word aligned. Except TPIU, ETM, and the ROM table, so become 0xE0042000 to 0xE000FF00.
Debug Access Port Bus	32-bit bus based on an enhanced version of the APB specification. This is for attaching debug interface blocks such as SWJ-DP or SWDP.

Typical Connections

Simple peripherals can be connected to the Cortex-M3 via an AHB-to-APB bridge.

Reset Signals

Power on reset	Reset that should be asserted when the device is powered up; resets both processor core and debugging system
System Reset	System reset; affects processor core, NVIC (except debug control registers), and MPU but not the debugging system
Test reset	Reset for debugging system

7. Exceptions

7.1 Exception Types

system exception: 1~15
External interrupt input: 16~255 (mapped to 0~239 IRQ)

The value of the current running exception is indicated by the special register IPSR or from the NVIC's Interrupt Control State Register (the VECTACTIVE field).

7.2 Definitions of Priority

A higher-priority (smaller number in priority level) exception can preempt a lower-priority (larger number in priority level) exception; this is the nested exception/interrupt scenario.

256 levels of programmable priority (a maximum of 128 levels of preemption).

If 3 bit,

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Implemented			Not implemented, read as zero				

If 4 bit,

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Implemented				Not implemented, read as 0			

8bit divide into two parts: preempt priority and subpriority.

Priority Group in NVIC

The preempt priority level	defines whether an interrupt can take place when the processor is already running another interrupt handler.
The subpriority level value	is used only when two exceptions with same preempt priority level occur at the same time.

Priority Group	Preempt Priority Field	Subpriority Field
0	Bit [7:1]	Bit [0]
1	Bit [7:2]	Bit [1:0]
2	Bit [7:3]	Bit [2:0]
3	Bit [7:4]	Bit [3:0]
4	Bit [7:5]	Bit [4:0]
5	Bit [7:6]	Bit [5:0]
6	Bit [7]	Bit [6:0]
7	None	Bit [7:0]

Bits	Name	Type	Reset Value	Description
31:16	VECTKEY	R/W	–	Access key; 0x05FA must be written to this field to write to this register, otherwise the write will be ignored; the read-back value of the upper half word is 0xFA05
15	ENDIANNESS	R	–	Indicates endianness for data: 1 for big endian (BE8) and 0 for little endian; this can only change after a reset
10:8	PRIGROUP	R/W	0	Priority group
2	SYSRESETREQ	W	–	Requests chip control logic to generate a reset
1	VECTCLRACTIVE	W	–	Clears all active state information for exceptions; typically used in debug or OS to allow system to recover from system error (Reset is safer)
0	VECTRESET	W	–	Resets the Cortex-M3 processor (except debug logic), but this will not reset circuits outside the processor

Application Interrupt and Reset Control Register (Address 0xE000ED0C)

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Preempt priority		Sub priority					

7.3 Vector Tables

By default, the vector table starts at address zero.
the vector table can be relocated to other memory locations.
the *vector table offset register* (address 0xE000ED08).

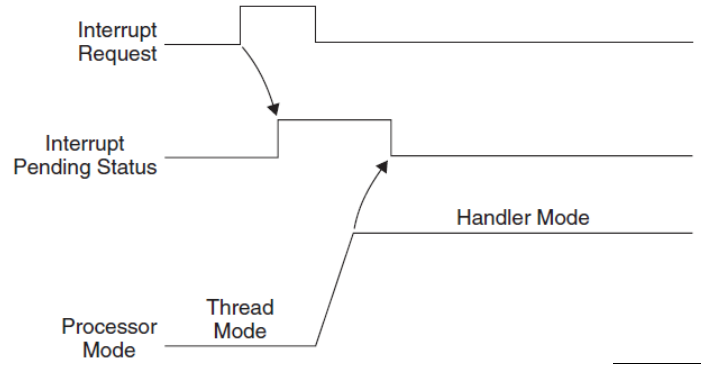
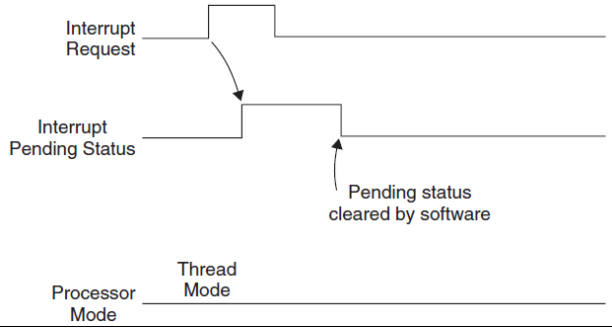
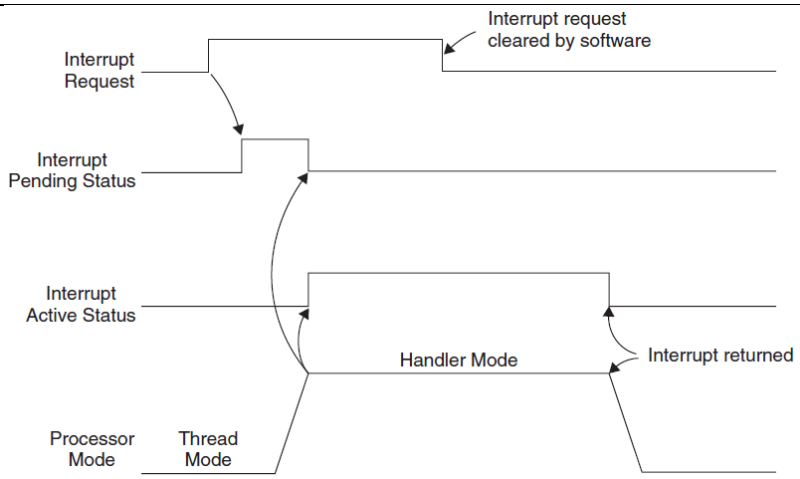
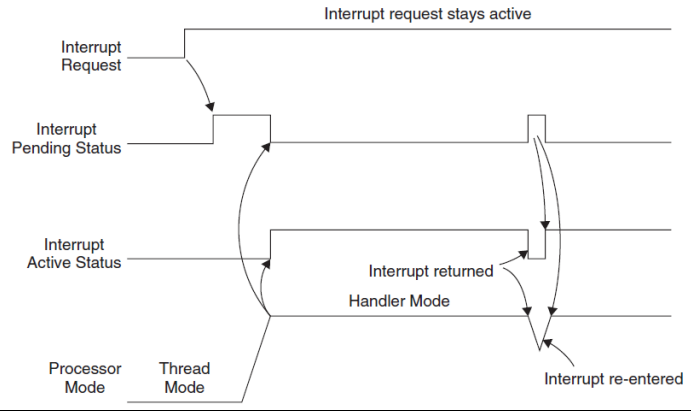
Bits	Name	Type	Reset Value	Description
29	TBLBASE	R/W	0	Table base in Code (0) or RAM (1)
28:7	TBLOFF	R/W	0	Table offset value from Code region or RAM region

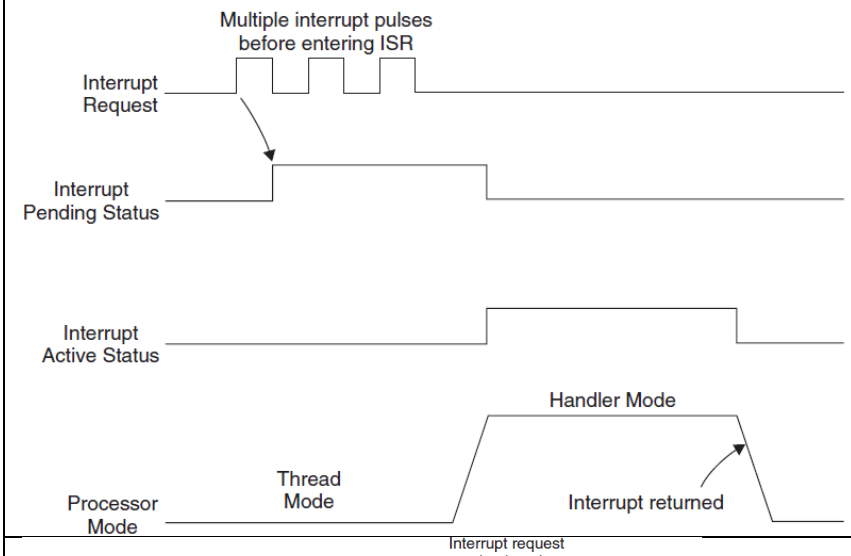
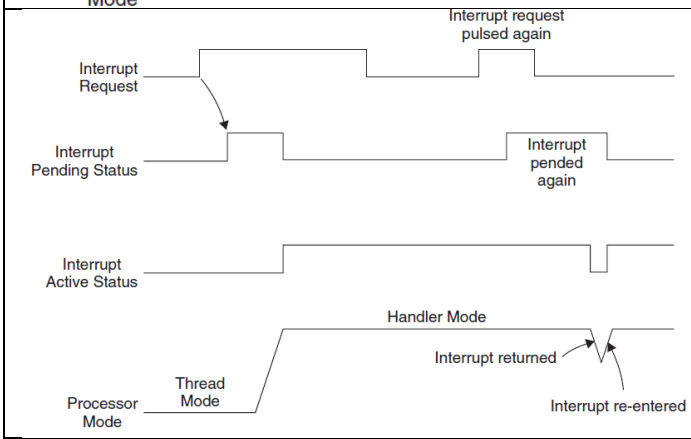
Vector Table Offset Register (Address 0xE000ED08)
On new vector table, it minimum need, MSP,ResetVector,NMI vector, and Hard fault vector.

7.4 Interrupt Inputs and Pending Behavior

Think interrupt as 3 items, Request, Pending Status and Handler.

case study	Explain

	When interrupt input is asserted, it becomes a Pending Status. Pended interrupt will executes the interrupt handler.
	If Pending status register is cleared (PRIMASK/FAULTMASK was 1), interrupt handler is canceled. The pending status of the interrupt can be accessed in the NVIC and is writable, so you can clear a pending interrupt or use software to pend a new interrupt by setting the pending register.
	When interrupt handler becomes Active, Pending status is cleared. When an interrupt is active, you cannot start processing the same interrupt again until the interrupt service routine is terminated with an interrupt return.
	If Interrupt request is high, it can be re-Pending status.

	If interrupt is pulsed, before the processor starts processing it, it will be treated as one single interrupt request
	If an interrupt is de-asserted and then pulsed again during the interrupt service routine, it will be pended again.
Note: Pending of an interrupt can happen even if the interrupt is disabled; the pended interrupt can then trigger the interrupt sequence when the enable is set later. As a result, before enabling an interrupt, it could be useful to check whether the pending register has been set. The interrupt source might have been activated previously and have set the pending status. If necessary, you can clear the pending status before you enable an interrupt.	

7.5 Fault Exception.

Fault is a high priority interrupt

	Description
Bus faults	Error response is received during a transfer on the AHB interfaces. Stacking error, unstacking error. Reading of an interrupt vector table If 'bus fault handler' is not enabled, the hard fault handler will be executed! Bus Fault Status Register (BFSR). Bus Fault Address Register (BFAR).

<table>
<tr><td></td><td>
<table>
<tr><th>Bits</th><th>Name</th><th>Type</th><th>Reset Value</th><th>Description</th></tr>
<tr><td>7</td><td>BFARVALID</td><td>–</td><td>0</td><td>Indicates BFAR is valid</td></tr>
<tr><td>6:5</td><td>–</td><td>–</td><td>–</td><td>–</td></tr>
<tr><td>4</td><td>STKERR</td><td>R/Wc</td><td>0</td><td>Stacking error</td></tr>
<tr><td>3</td><td>UNSTKERR</td><td>R/Wc</td><td>0</td><td>Unstacking error</td></tr>
<tr><td>2</td><td>IMPREISERR</td><td>R/Wc</td><td>0</td><td>Imprecise data access violation</td></tr>
<tr><td>1</td><td>PRECISERR</td><td>R/Wc</td><td>0</td><td>Precise data access violation</td></tr>
<tr><td>0</td><td>IBUSERR</td><td>R/Wc</td><td>0</td><td>Instruction access violation</td></tr>
</table>
Bus Fault Status Register (0xE000ED29)
Enable: BUSFAULTENA
</td></tr>
<tr><td>Memory management faults</td><td>
memory accesses that violate the setup in the MPU or by certain illegal accesses.
- Access to memory regions not defi ned in MPU setup
- Writing to read-only regions
- An access in the user state to a region defi ned as privileged access only
<table>
<tr><th>Bits</th><th>Name</th><th>Type</th><th>Reset Value</th><th>Description</th></tr>
<tr><td>7</td><td>MMARVALID</td><td>–</td><td>0</td><td>Indicates the MMAR is valid</td></tr>
<tr><td>6:5</td><td>–</td><td>–</td><td>–</td><td>–</td></tr>
<tr><td>4</td><td>MSTKERR</td><td>R/Wc</td><td>0</td><td>Stacking error</td></tr>
<tr><td>3</td><td>MUNSTKERR</td><td>R/Wc</td><td>0</td><td>Unstacking error</td></tr>
<tr><td>2</td><td>–</td><td>–</td><td>–</td><td>–</td></tr>
<tr><td>1</td><td>DACCVIOL</td><td>R/Wc</td><td>0</td><td>Data access violation</td></tr>
<tr><td>0</td><td>IACCVIOL</td><td>R/Wc</td><td>0</td><td>Instruction access violation</td></tr>
</table>
Memory Management Fault Status Register (0xE000ED28)
Enable: MEMFAULTEN
</td></tr>
<tr><td>Usage faults</td><td>
Undefined instructions
Coprocessor instructions
Trying to switch to the ARM state
Invalid interrupt return
Unaligned memory accesses
Divide by zero
<table>
<tr><th>Bits</th><th>Name</th><th>Type</th><th>Reset Value</th><th>Description</th></tr>
<tr><td>9</td><td>DIVBYZERO</td><td>R/Wc</td><td>0</td><td>Indicates a divide by zero has taken place (can be set only if DIV_0_TRP is set)</td></tr>
<tr><td>8</td><td>UNALIGNED</td><td>R/Wc</td><td>0</td><td>Indicates that an unaligned access fault has taken place</td></tr>
<tr><td>7:4</td><td>–</td><td>–</td><td>–</td><td>–</td></tr>
<tr><td>3</td><td>NOCP</td><td>R/Wc</td><td>0</td><td>Attempts to execute a coprocessor instruction</td></tr>
<tr><td>2</td><td>INVPC</td><td>R/Wc</td><td>0</td><td>Attempts to do an exception with a bad value in the EXC_RETURN number</td></tr>
<tr><td>1</td><td>INVSTATE</td><td>R/Wc</td><td>0</td><td>Attempts to switch to an invalid state (e.g., ARM)</td></tr>
<tr><td>0</td><td>UNDEFINSTR</td><td>R/Wc</td><td>0</td><td>Attempts to execute an undefined instruction</td></tr>
</table>
Usage Fault Status Register (0xE000ED2A)
Enable: USGFAULTENA
</td></tr>
<tr><td>Hard faults</td><td>The hard fault handler can be caused by usage faults, bus faults, and memory management faults if their handler cannot be executed.</td></tr>
</table>

Bits	Name	Type	Reset Value	Description
31	DEBUGEVT	R/Wc	0	Indicates hard fault is triggered by debug event
30	FORCED	R/Wc	0	Indicates hard fault is taken because of bus fault, memory management fault, or usage fault
29:2	–	–	–	–
1	VECTBL	R/Wc	0	Indicates hard fault is caused by failed vector fetch
0	–	–	–	–

Hard Fault Status Register (0xE000ED2C)

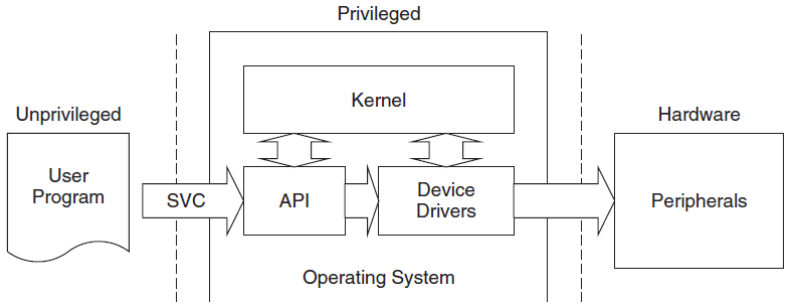
To enable, Enable flag on System Handler Control and State register in NVIC

Dealing with Faults

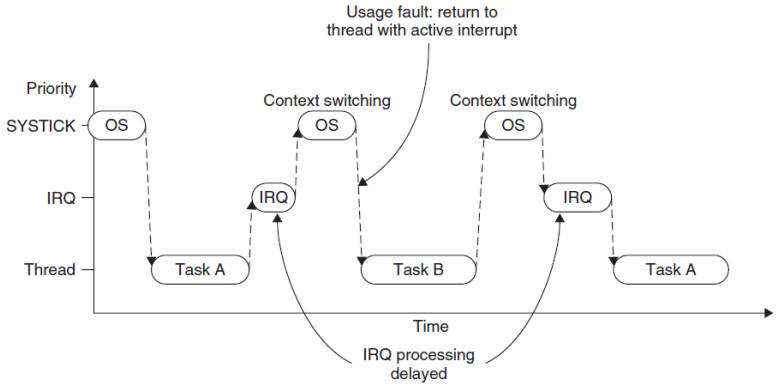
During software development, we can use the Fault Status Registers (FSRs) to determine the causes of errors in the program and correct them.

7.6 SVC and PendSV

SVC: System Service Call. for generating system function calls
PendSV: Pended System Call

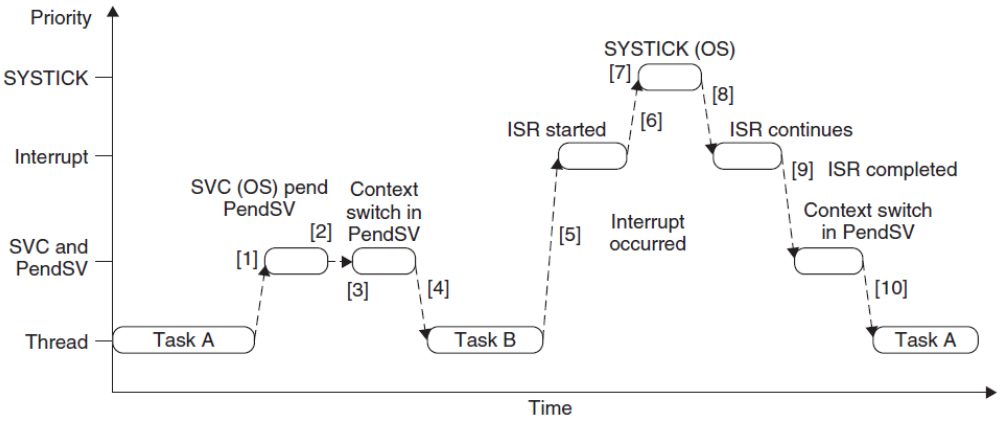


If the system uses a PSP for user applications, you might need to determine which stack was used first. This can be determined from the link register value when the handler is entered.
PendSV can be pended.



Problem with Context Switching at the IRQ.

The PendSV exception solves the problem by delaying the context-switching request until all other IRQ handlers have completed their processing. To do this, the PendSV is programmed as the lowest priority exception. If the OS detects that an IRQ is currently active (IRQ handler running and preempted by SYSTICK), it defers the context switching by pending the PendSV exception.



8. NVIC and Interrupt Control

integrated part of processor.
include the control registers and control logic for interrupt processing.
include control registers for MPU, SYSTICK and debugging control.
location 0xE000E000
MRS and MSR instruction

Interrupt Enable / Clear Enable

SETENA To set the enable bit
CLRENA To clear the enable bit
: will not affect other interrupt enable states.
: first 16 is for exception, interrupt# starts from 16.

0xE000E100~ SETENAx
0xE000E180~ CLRENAx

Interrupt Pending and Clear Pending

Interrupt Happen -> becomes Pending status.
Access
SETPEND : Interrupt Set Pending
CLRPEND: Interrupt Clear Pending

0xE000E200~ SETPENDx
0xE000E280~ CLEPENDx

Priority Levels

8~3bits, can be divided into preempt priority level and subpriority level.

0xE000E400~ PRI_x
~0xE000E4EF

Active Status

When the processor starts the interrupt handler, the bit is set to 1 and cleared when the interrupt return is executed.
Interrupt can be preempted by higher interrupt.

0xE000E300~ ACTIVEx

PREMASK and FAULTMASK special Registers

PRIMASK to disable all exceptions except NMI and hard fault

is useful for temporarily disabling all interrupts for critical tasks.

FAULTMASK it changes the effective current priority level to "-1" so that even the hard fault handler is blocked.
Only the NMI can be executed when FAULTMASK is set.
FAULTMASK is cleared automatically upon exiting the exception handler.

Exception Number	Exception Type	Priority (Default to 0 if Programmable)	Description
0	NA	NA	exception running
1	Reset	-3 (Highest)	Reset
2	NMI	-2	Nonmaskable interrupt. (external NMI input)
3	Hard fault	-1	All fault conditions, if the corresponding fault handler is not enabled
..	..	..	..

The BASEPRI Special Register

to disable interrupts only with priority lower than a certain level.
simply write the required masking priority level to the BASEPRI register.
To cancel the masking, just write 0 to the BASEPRI register:
BASEPRI_MAX register, for Assembler.

```
MOV R0, #0x60
MSR BASEPRI_MAX, R0 ; Disable interrupts with priority
                    ; 0x60, 0x61,..., etc
MOV R0, #0xF0
MSR BASEPRI_MAX, R0 ; This write will be ignored because
                    ; it is lower
                    ; level than 0x60
MOV R0, #0x40
MSR BASEPRI_MAX, R0 ; This write is allowed and change the
                    ; masking level to 0x40
```

Configuration Register for Other Exceptions

The pending status of faults and active status of most system exceptions are also available from this register

0xE000ED24 System Handler Control and Status Register

Bits	Name	Type	Reset Value	Description
18	USGFAULTENA	R/W	0	Usage fault handler enable
17	BUSFAULTENA	R/W	0	Bus fault handler enable
16	MEMFAULTENA	R/W	0	Memory management fault enable
15	SVCALLPENDED	R/W	0	SVC pended; SVCall was started but was replaced by a higher-priority exception
14	BUSFAULTPENDED	R/W	0	Bus fault pended; bus fault handler was started but was replaced by a higher-priority exception
13	MEMFAULTPENDED	R/W	0	Memory management fault pended; memory management fault started but was replaced by a higher-priority exception
12	USGFAULTPENDED	R/W	0	Usage fault pended; usage fault started but was replaced by a higher-priority exception
11	SYSTICKACT	R/W	0	Read as 1 if SYSTICK exception is active
10	PENDSVACT	R/W	0	Read as 1 if PendSV exception is active
8	MONITORACT	R/W	0	Read as 1 if debug monitor exception is active
7	SVCALLACT	R/W	0	Read as 1 if SVCall exception is active
3	USGFAULTACT	R/W	0	Read as 1 if usage fault exception is active
1	BUSFAULTACT	R/W	0	Read as 1 if bus fault exception is active
0	MEMFAULTACT	R/W	0	Read as 1 if memory management fault is active
Note: Bit 12 (USGFAULTPENDED) is not available on revision 0 of Cortex-M3.				

0xE000ED04 Interrupt Control and Status register.
Pending for NMI, the SYSTICK timer, and PendSV is programmable by it.

Bits	Name	Type	Reset Value	Description
31	NMIPENDSET	R/W	0	NMI pended
28	PENDSVSET	R/W	0	Write 1 to pend system call Read value indicates pending status
27	PENDSVCLR	W	0	Write 1 to clear PendSV pending status
26	PENDSTSET	R/W	0	Write 1 to pend SYSTICK exception Read value indicates pending status
25	PENDSTCLR	W	0	Write 1 to clear SYSTICK pending status
23	ISRPREEMPT	R	0	Indicates that a pending interrupt is going to be active in the next step (for debug)
22	ISRPENDING	R	0	External interrupt pending (excluding system exceptions such as NMI for fault)
21:12	VECTPENDING	R	0	Pending ISR number
11	RETTOBASE	R	0	Set to 1 when the processor is running an exception handler; will return to Thread level if interrupt return and no other exceptions pending
9:0	VECTACTIVE	R	0	Current running interrupt service routine

0xE000E004 Interrupt Controller Type register
gives the number of interrupt inputs supported, in granularities of 32

Example Procedures in Setting Up an Interrupt

Example procedure for setting up an interrupt:
1. By default the priority group 0 is used
2. Copy the hard fault and NMI handlers to a new vector table location if vector table relocation is required.
3. The Vector Table Offset register should also be set up to get the vector table ready
4. Set up the interrupt vector for the interrupt
5. Set up the priority level for the interrupt
6. Enable the interrupt

Software Interrupts

method1: SETPEND register
method2: STIR Software Trigger Interrupt Register

0xE000EF00 STIR. ex: write 0 to pend external interrupt #0.

By default, a user program cannot write to the NVIC; however, if it is necessary for a user program to write to this register, the bit 1 (USERSETMPEND) of the NVIC Confi guration Control Register (0xE000ED14) can be set to allow user access to the NVIC's STIR.

The SYSTICK Timer

SYSTICK Timer is integrated with the NVIC.
24-bit down counter.
input: FCLK or STCLK.

0xE000E010 SYSTICK Control and Status Register
0xE000E014 SYSTICK Reload Value Register
0xE000E018 SYSTICK Current Value Register
0xE000E01C SYSTICK Calibration Value Register

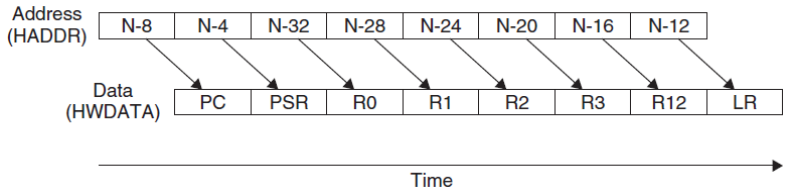
9 Interrupt Behavior

9.1 Interrupt/Exception Sequences

Stacking
Vector fetch
Update of the stack pointer, link registers and program counter

9.1.1 Stacking

PC, PSR, R0–R3, R12, and LR are pushed to the stack (to PSP or MSP). Afterward, the main stack will always be used during the handler, so all nested interrupts will use the main stack.



Address	Data	Push Order
Old SP (N) ->	(Previously pushed data)	-
(N-4)	PSR	2
(N-8)	PC	1
(N-12)	LR	8
(N-16)	R12	7
(N-20)	R3	6
(N-24)	R2	5
(N-28)	R1	4
New SP (N-32) ->	R0	3

9.1.2 Vector Fetches

It fetches the exception vector (the starting address of the exception handler) from the vector table

9.1.3 Register Updates

	Description
SP	The Stack Pointer (either the MSP or the PSP) will be updated to the new location during stacking. During execution of the interrupt service routine, the MSP will be used if the stack is accessed.
PSR	The IPSR (the lowest part of the PSR) will be updated to the new exception number.
PC	This will change to the vector handler as the vector fetch completes and starts fetching instructions from the exception vector.
LR	The LR will be updated to a special value called EXC_RETURN. This special value drives the interrupt return operation. The last 4 bits of the LR have a special meaning, which is covered later in this chapter.

EXC_RETURN has values with bit[31:4] and are all 1 (i.e., 0xFFFFFFFX); the last 4 bits define the return information.

9.2 Exception Exits

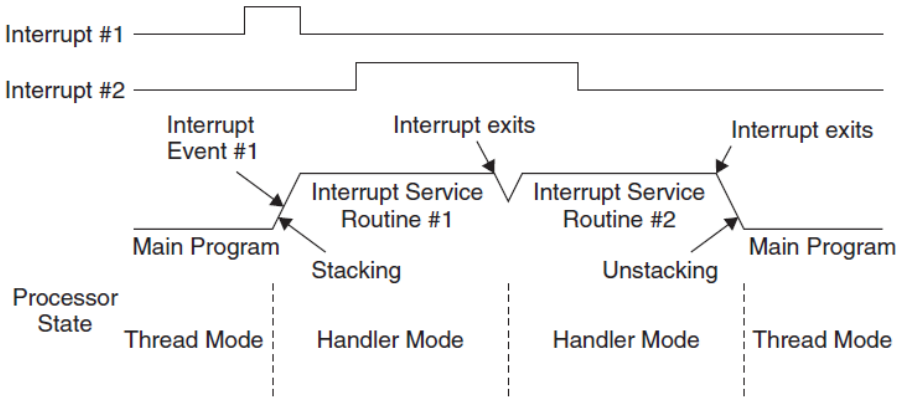
Return Instruction	Description
BX <*reg*>	If the EXC_RETURN value is still in LR, we can use the *BX LR* instruction to perform the interrupt return.
POP {PC}, or POP {...., PC}	Very often the value of LR is pushed to the stack after entering the exception handler. We can use the POP instruction, either a single POP or multiple POPs, to put the EXC_RETURN value to the program counter. This will cause the processor to perform the interrupt return.
LDR, or LDM	It is possible to produce an interrupt return using the LDR instruction with PC as the destination register.

9.3 Nested Interrupts

each nested interrupts uses 8 words on MSP. Need enough stack memory.
Reentrance exceptions are not allowed.
SVC instructions cannot be used inside an SVC handler.

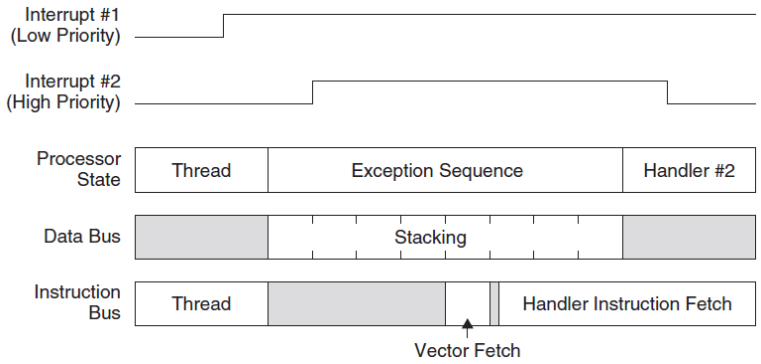
9.4 Tail-Chaining Interrupts

skip un-stacking and stacking.



9.5 Late Arrivals

Higher exception is arrived on the stacking stage, it executes the high priority exception.



9.6 More on the Exception Return Value

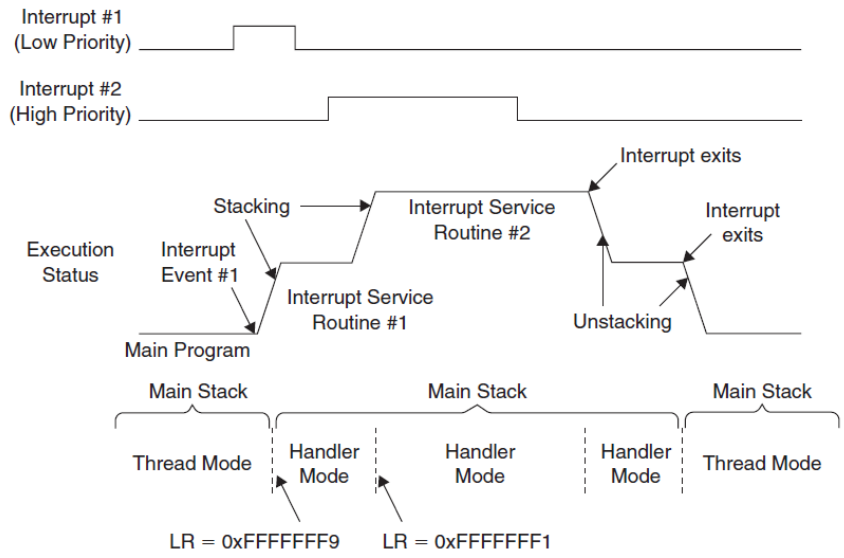
LP[31:4] are all 1. EXC_RETURN. and bit[3:0] provides information required by the exception return operation

Bits	31:4	3	2	1	0
Descriptions	0xFFFFFFF	Return mode (Thread/handler)	Return stack	Reserved; must be 0	Process state (Thumb/ARM)

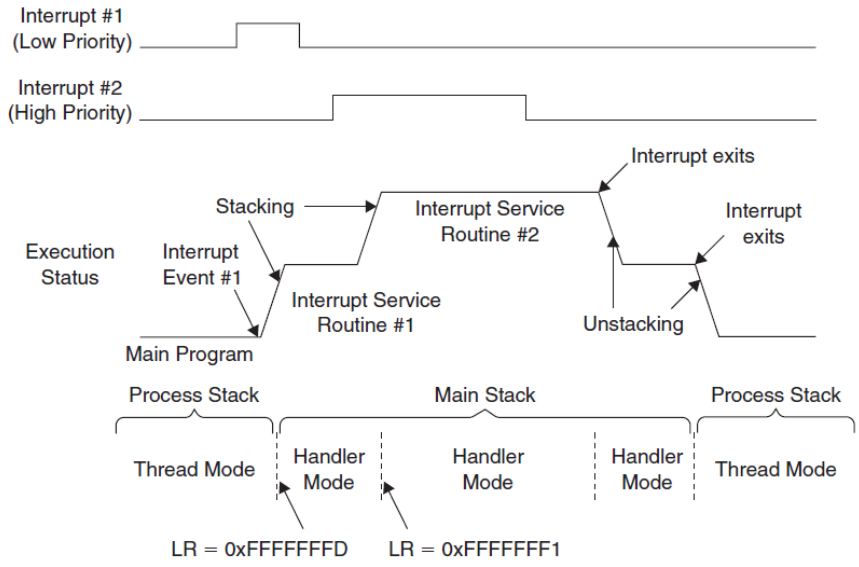
Value	Condition
0xFFFFFFF1	Return to handler mode
0xFFFFFFF9	Return to Thread mode and on return use the main stack
0xFFFFFFFD	Return to Thread mode and on return use the process stack

0001b - when a nested exception is entered
1001b - If the thread is using the MSP (main stack),
1101b - If the thread is using PSP (process stack),

If the thread is using the MSP (main stack), the value of LR will be set to 0xFFFFFFF9 when it enters an exception, and 0xFFFFFFF1 when a nested exception is entered.



If the thread is using PSP (process stack), the value of LR would be 0xFFFFFFFD when entering the first exception and 0xFFFFFFF1 for entering a nested exception



9.7 Interrupt Latency

12 cycles = stacking (8)+ vector fetch + fetching instruction

tail-chaining: 6 cycles

9.8 Faults Related to Interrupts

Stacking

10. Cortex-M3 Programming

```
#define NVIC_CCR ((volatile unsigned long *)(0xE000ED14))
*NVIC_CCR = *NVIC_CCR | 0x200; /* Set STKALIGN */
```

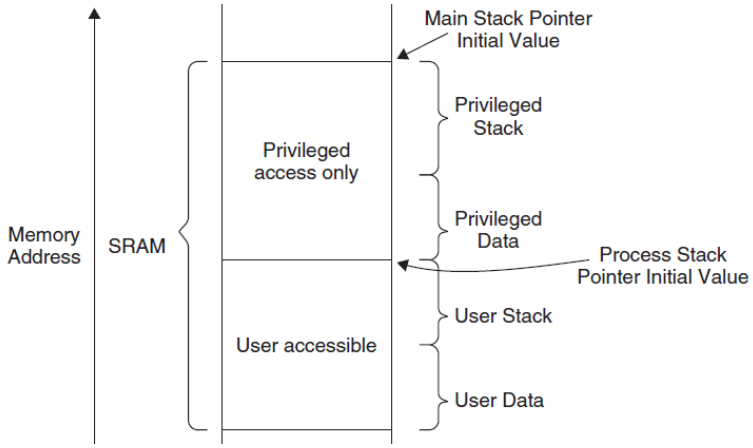
For simple cases, when a calling program needs to pass parameters to a subroutine or function, it will use registers R0 to R3, where R0 is the fi rst parameter, R1 is the second, and so on. Similarly, R0 is used for returning a value at the end of a function.

11. Exceptions Programming

- Stack setup
- Vector table setup
- Interrupt priority setup
- Enable the interrupt

When determining the stack size required, besides checking the stack level that could be used by the software, you also need to check how many levels of nested interrupts can occur.
For each level of nested interrupts, you need at least eight words of stack.

12 Advanced Programming Features and System Behavior



13. The Memory Protection Unit

Bits	Name	Type	Reset Value	Description
23:16	IREGION	R	0	Number of instruction regions supported by this MPU; because ARM v7-M architecture uses a unified MPU, this is always 0
15:8	DREGION	R	0 or 8	Number of regions supported by this MPU; in the Cortex-M3 this is either 0 (MPU not present) or 8 (MPU present)
0	SEPARATE	R	0	This is always 0 as the MPU is unified

MPU Type Register (0xE000ED90)

Bits	Name	Type	Reset Value	Description
2	PRIVDEFENA	R/W	0	Privileged default memory map enable. When set to 1 and if the MPU is enabled, the default memory map will be used for privileged accesses as a background region. If this bit is not set, the background region is disabled and any access not covered by any enabled region will cause a fault.
1	HFNMIENA	R/W	0	If set to 1, it enables the MPU during the hard fault handler and NMI handler; otherwise, the MPU is not enabled for the hard fault handler and NMI.
0	ENABLE	R/W	0	Enables the MPU if set to 1.

MPU Control Register (0xE000ED94)

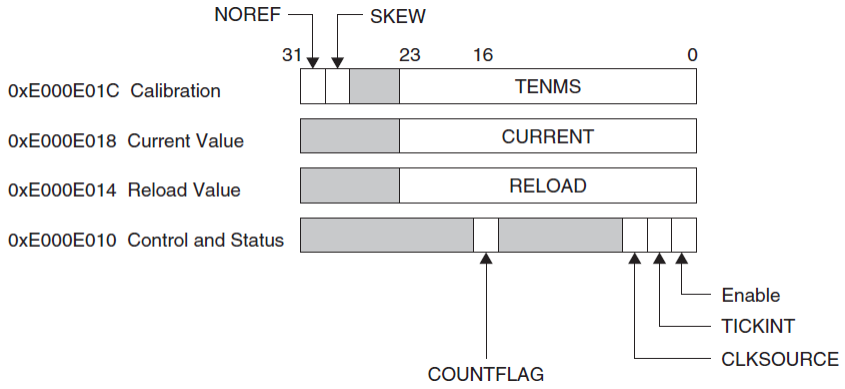
...

...

...

14 Other Cortex-M3 Features

The SYSTICK Timer



15 Debug Architecture

16 Debugging Components

Debug component

• Fetch Patch and BreakPoint Unit (FPB) • Data WatchPoint and Trace Unit (DWT)

• Instrumentation Trace Macrocell (ITM) • Embedded Trace Macrocell (ETM) • Trace Port Interface Unit (TPIU) • ROM Table

Debugging Support

Processor has debugging features such as program execution controls, including halting and stepping, instruction breakpoints, data watchpoints, registers and memory accesses, profi ling, and traces.

Debugging hardware is based on CoreSight architecture.

Debug Port	Debug Access Port (DAP). Dos not have a JTAG interface.
SWJ-DP	Serial Wire protocol and JTAG protocol
SW-DP	Support the Serial Wire protocol only
JTAP-DP	from ARM CoreSight product

an Embedded Trace Macrocell (ETM) to allow instruction trace. Trace information is output via the Trace Port Interface Unit (TPIU), and the debug host (usually a PC) can then collect the executed instruction information via external trace-capturing hardware.

The data watchpoint function is provided by a Data Watchpoint and Trace (DWT) unit
Flash Patch and Breakpoint (FPB) unit

GDB

Command	Request Format	Response Format
Read registers	g	*data*
Write registers	G*data*	OK
Read data at address	m*address*,*length*	*data*
Write data at address	M*address*,*length*:*data*	OK
Start/restart execution	c	S*signal*
Start execution from address	c*address*	S*signal*
Single step	s	S*signal*
Single step from address	s*address*	S*signal*
Reset/kill program	k	*no response*